

CSAW VIP 2025 Final Paper

Tony Tanory
Georgia Institute of Technology
Atlanta, Georgia
ttanory3@gatech.edu

Lindsay Lane Estrella
Georgia Institute of Technology
Atlanta, Georgia
lestrella7@gatech.edu

David Kim
Georgia Institute of Technology
Atlanta, Georgia
dkim944@gatech.edu

Kian Esmaeili
University of North Georgia
Atlanta, Georgia
kesma9557@ung.edu

Abstract—This report presents the semester long progression of work completed in preparation for and participation in the CSAW Embedded Security Challenge (ESC). The project combined hardware-based fault injection and side-channel analysis with exploratory research into the use of large-language models (LLMs) and AI-assisted tooling for cybersecurity workflows. Over the course of the semester, the work encompassed environment setup and toolchain stabilization, in-depth analysis of the challenges, successful fault-injection exploitation of GateKeeper, Critical Calculation, Sorters Song and early exploration of the other released challenges. Alongside hands-on technical experimentation, significant attention was devoted to assessing the feasibility of AI/LLM integration for code understanding, automation, trace analysis, and debugging support. Together, these efforts establish a comprehensive foundation for understanding hardware-centric attack surfaces.

I. INTRODUCTION

The CSAW Embedded Security Challenge represents an opportunity for exploring hardware vulnerabilities, physical attack surfaces, and embedded-system behavior under conditions such as side-channel leakage and fault injections. Attempting the competition required the establishment of a robust hardware and software environment, including configuring the ChipWhisperer-Nano platform, stabilizing the Jupyter-based capture and analysis workflow, and building familiarity with power-analysis, timing-analysis, and fault-injection methodologies. Early in the semester, this technical foundation was paired with an independent research effort into the application of AI and large-language models to embedded security tasks. This research surveyed potential uses of LLMs for firmware summarization, script generation, trace-pattern extraction, and anomaly detection.

As the semester progressed, this groundwork was applied to the challenges. Work began with GateKeeper’s Revenge and Gatekeeper, where the focus centered on understanding password-comparison logic, leakage sources, early-exit timing behavior, and the role of external triggers. Subsequent stages involved refining attack strategies, documenting information-flow vulnerabilities, and exploring how AI-assisted analysis could support scripting and diagnostic reasoning. Newer challenges released later in the semester were also reviewed. Col-

lectively, these efforts reflect a semester-long progression from foundational setup through complex exploitation and reflective analysis of AI’s role within embedded security research.

II. GATEKEEPER’S REVENGE

A. Challenge Objective

GateKeeper’s Revenge served as an introductory and qualifying challenge designed to transition competitors into the required mindset for the upcoming challenges. The objective was not to fully solve the challenge but rather prompt participants to analyze the provided information, identify weak points and construct a plan for how an attack would be executed.

B. Code Implications

The verifier leaks information through three specific code behaviors:

- 1) Early exit on mismatch: The routine stops at the first nonmatching byte, so iterations (and time) scale with the correct prefix length (*if (data[i] != password[i]) break;*)
- 2) Per-math busy-wait delay: Each matched byte incurs a fixed delay, adding a distinctive repetitive pattern and longer activity under the trigger (*for (volatile uint32_t i = 0; i < 5000; i++);*)
- 3) External trigger around the compare loop: A trigger is raised before the loop and lowered after, clearly marking the sensitive region for alignment and analysis (*trigger_high() / trigger_low()*)

These mechanisms show in both timing and power domains. Execution time increases with the number of correct prefix bytes, while power traces reveal repetitive patterns corresponding to each successful comparison. Visual inspection shows one trace with a strong periodic pattern and longer activity under the trigger. This is consistent with a match-induced delay, suggesting the correct first character is likely the one exhibiting that pattern.

C. Attack Method

A structured evaluation plan combined measurement, statistical analysis, and resilience testing:

- Random guesses can be sent to capture power traces and clock timing. Then, the average timing vs. number of matching prefix bytes to confirm the linear relation can be computed.
- Because we know the routine breaks on mismatch, it is safe to determine how many leading bytes are correct for any given guess by measuring the response time or energy.
- This method can be used to iteratively build the password by setting all bytes except position k to a known prefix (the bytes already discovered) and trying all 256 values for byte k . The value that results in longer execution should mean that the byte k is correct.

Another way that can be done is with fault injection.

- Voltage glitching can be used to skip the break or skip directly to the increment of *matched_chars*.
- We can cause the loop to continue past a mismatch to allow *matched_chars* to become equal to *password_len*. The device will return *correct=1* if it believes that the password is matched.
- The goal here is to make the device think the comparisons succeeded by using fault injection to reveal intermediate values by causing an exception that returns data over a debug.

III. GATEKEEPER CHALLENGE SET

A. Challenge Objective

The GateKeeper challenge subproblem aims to recover two secret authentication tokens. Command *a* (13-byte *gk1...*) and command *b* (17-byte *gk2...*). The firmware performs a byte-by-byte comparison and introduces a secret-dependent delay for each matched byte. The objectives for this challenge set is to demonstrate and document how the leakage can be observed and recover the full secret strings.

B. Tools

- ChipWhisperer Capture software and Python API
- Jupyter Notebook (for automation and data analysis)
- NumPy for trace processing and statistical operations
- Oscilloscope integrated within ChipWhisperer for power trace capture
- ChatGPT for script automation

C. Gatekeeper 1 Recovery

During static review the team found an interesting source code comment that was believed to be the password for *gk1*. The comment was used verbatim as a *gk1* candidate, flashed on the ChipWhisperer NANO, and submitted to the device's verification routine. The simple serial output of the firmware confirmed the password. This bypass required no side channel analysis. However, in order to receive points for the flag the team needed to confirm the flag via a timing exploitation.

Gatekeeper 1

```
# Gatekeeper 1: 8 character flag, lowercase alpha or numeric
target.simpleserial_write('a', b'gk1{l0g1npwn}') #l0g1npwn
response = target.simpleserial_read('r', 1)
if response is not None:
    print(f"Gatekeeper1 Success (0x01 if correct): {response.hex()}")

target.simpleserial_write('q', b'0')
response = target.simpleserial_read('r', 4)
if response is not None:
    print(f"Number of Queries (cumulative): {int.from_bytes(response, 'big')}")

Gatekeeper1 Success (0x01 if correct): 01
Number of Queries (cumulative): 3
```

Fig. 1. Gatekeeper 1 flag confirmation.

D. Gatekeeper 2 Recovery

For the second Gatekeeper flag, we attempted a timing attack analyzing the time to process each character, exploiting the for loop in the *gatekeeper.c* file.

```
uint32_t j= 0; j < (uint32_t) (5000 /
(c+1)) - i*125; j++);
```

Theoretically, if you submit candidate bytes for a position in the flag while having the earlier bytes correct, the response time will be measurably larger when the candidate is correct. However, in practice after running a sample script, the difference in response time is within milliseconds and has very high variance. The team ran a timing comparison script that ran a baseline filler password against a candidate password we had derived from the binary, this resulted in no notable difference in timing. Next, the team pivoted to running a script that tests different ASCII values and picking the byte that produces the largest average response time. Essentially, the more matched bytes the longer the delay should be. However, our results did not yield anything useful as every character tested was within 0.0001s of each other.

```
[*]Recovering pos 0 (0-based inside
the braces). Known so far:????????
->best char: 'N' avg_time: 0.0012s
Top 5: [('N', 0.0012247562408447266),
('L', 0.0012189547220865886),
('Q', 0.0012097358703613281),
('P', 0.0011879603068033855),
(' ', 0.0011753241221110027)]
```

Clearly, this approach was not the correct one.

IV. SORTERS SONG CHALLENGE SET

A. Challenge Objective

The Sorter's Song firmware implements two insertion-sort routines:

- *sort8()* sorts 15 one-byte values used in Flag 1.
- *sort16()* sorts 15 two-byte values used in Flag 2.

The flags are transmitted only when the submitted arrays match the internally generated sorted arrays `original_arr1` and `original_arr2`. These arrays are never sent over UART and are regenerated once per power cycle. The main vulnerability is that the sorting routine's execution time and power consumption depend on the number of shifts performed, which in turn depend on how many array elements are smaller than the current input. This behavior creates a measurable timing side-channel.

B. Tools

- ChipWhisperer Capture software and Python API
- Jupyter Notebook (for automation and data analysis)
- NumPy for trace processing and statistical operations
- Oscilloscope integrated within ChipWhisperer for power trace capture

C. Sorters Song 1 Recovery

The first challenge used a byte array (`uint8_t`). Because all values are unique, each insertion caused either 0 or 1 additional shift when compared to a correctly sorted prefix. We leveraged this property to recover *original_arr1* byte by byte from right to left.

- 1) Reset device and initialize *original_arr1*.
- 2) Construct test input: `data = [X] + original_arr1[k..]` using the SimpleSerial 'p' command with `skip = k`.
- 3) Run `sort8()` with the 'c' command while capturing power traces.
- 4) Compute a moving average envelope of the trace and find the point where the activity drops below the baseline.
- 5) Use durations for `X=0` and `X=255` to calibrate "0-shift" vs "1-shift." Binary-search `X` to find the smallest value producing a longer duration.
- 6) Repeat for `k = 14` to `0`.

As a result we recover the array [7, 12, 43, 52, 57, 66, 80, 104, 113, 124, 136, 147, 172, 177, 219], which gives us the flag `ss1{y0u_g0t_it_br0!}`.

D. Sorters Song 2 Recovery

For the next part of the Sorters Song challenge, we need to recover *original_arr2*, a 15-element sorted `uint16` array, from the same firmware using power traces from the 16-bit insertion sort (`sort16()`).

Unlike the 8-bit case, *original_arr2* may contain duplicate values, which invalidates simple "0 vs 1 shift" classification. Multiple identical values can cause multiple shifts even at the same boundary.

We modified the earlier attack to handle duplicates safely:

- 1) Reset and initialize *original_arr2*.
- 2) Probe at index `k`: Build `[X] + original_arr2[k..]`.
- 3) Capture sort trace using 'd' command and extract activity duration.
- 4) For `k=14`, Measure durations for `X=0` and `X=65535`. Then, binary search for smallest `X` with duration above threshold(`a[14] = X1`).

- 5) For `k ; 14`, Measure duration(`X=0`) and duration(`X=a[k+1]`). If durations are nearly equal, then `a[k] = a[k+1]`. Otherwise, binary-search `[0, a[k+1]]` for smallest `X` with longer duration (`a[k] = X1`).
- 6) Each recovered `uint16` gets packed into `[lo, hi]`, forming a 30-byte payload.

Code snippet for reference:

```
def recover_arr2(reps0=7, reps_mid=5, eq_eps=2):
    reset_device()
    a = [0]*15
    # Recover a[14]
    t0 = median_dur(0, 14, reps=reps0)
    t1 = median_dur(65535, 14, reps=reps0)
    thr = (t0 + t1)/2.0
    lo, hi = 0, 65535
    while lo < hi:
        mid = (lo+hi)//2
        d = median_dur(mid, 14, reps=reps_mid)
        hi = mid if d >= thr else (mid+1)
    a[14] = hi-1
    # Remaining indices
    for k in range(13, -1, -1):
        t0 = median_dur(0, k, reps=reps0)
        t_eq = median_dur(a[k+1], k, reps=reps0)
        if abs(t_eq - t0) <= eq_eps:
            a[k] = a[k+1]; continue
        thr = (t0 + t_eq)/2.0
        lo, hi = 0, a[k+1]
        while lo < hi:
            mid = (lo+hi)//2
            d = median_dur(mid, k, reps=reps_mid)
            hi = mid if d >= thr else (mid+1)
        a[k] = hi-1
    return a
```

This gives us an array, [102, 3, 138, 36, 34, 64, 1, 73, 225, 75, 92, 105, 156, 112, 137, 135, 135, 144, 246, 148, 114, 154, 58, 166, 101, 179, 22, 201, 219, 212], which then gives us the flag `ss2{!AEGILOPS_chimps}`.

E. Discussion

These attacks demonstrate that even simple algorithms like insertion sort can unintentionally leak sensitive information when their control flow depends on secret data values. In this challenge, performance was optimized by reducing the number of necessary captures per element—each array entry was typically recovered within around sixteen binary-search iterations, with five power traces collected per iteration, resulting in roughly four thousand total queries for a complete array reconstruction. The process was fully automated using Python notebooks that coordinated data acquisition, trace capture, and classification, relying on median-based duration estimation to

ensure consistency and noise tolerance across repeated runs. Additionally, large language models were leveraged throughout development to assist with debugging scripts, optimizing binary-search logic, refining signal-processing heuristics, and improving overall efficiency and reproducibility of the attack pipeline.

V. CRITICAL CALCULATION CHALLENGE SET

A. Challenge Objective

The Critical Calculation challenge required participants to analyze and exploit a diagnostic integrity-check routine embedded within the device’s firmware. Upon receiving the simpleserial command ‘d’, the target system raises a trigger signal and executes a deterministic double-nested loop totaling 8000 iterations. The firmware then evaluates whether the final counter value matches the expected constant. If the computation proceeds uninterrupted, the device outputs a padded *DIAGNOSTICOK* message. However, if the loop is disturbed by a carefully timed glitch, the post-diagnostic check instead leaks the secret flag. The objective of the challenge was to understand the timing characteristics of the diagnostic loop, identify a vulnerable glitch window, and manipulate execution in a way that bypasses the integrity check.

B. Tools

- ChipWhisperer Capture software and Python API. Used for configuring the CW-Nano, Jupyter notebook, controlling glitch parameters, capturing responses, and automating repeated diagnostic queries.
- NumPy for trace processing and statistical operations
- ChipWhisperer’s integrated oscilloscope used to monitor trigger timing, observe diagnostic-loop execution windows, and align glitch offsets with cycle boundaries.

C. Critical Calculation Recovery

The flag recovery was done by iterative refinement of the glitch parameters. Initial baseline observations confirmed the stability of the diagnostic output under non-glitched conditions, allowing clear visibility of abnormal behavior when faults were successfully introduced. Early glitch attempts proved ineffective. Often either missing the vulnerable region or causing complete target destabilization. By progressively narrowing the glitch offset window, which was done by aligning it to the midpoint of the loop, and adjusting the glitch width to a range sufficient to disrupt a single iteration but not lock the device, a stable fault was eventually induced. This caused the internal counter to deviate from its expected value, which ended up triggering the alternate code path and returning the hidden flag through the simpleserial interface.

VI. INDIVIDUAL CONTRIBUTIONS

A. Kian Esmaeili

Focused heavily on foundational and applied research for the CSAW qualification challenge. He conducted deep technical research on a range of side-channel and fault-injection attacks including timing, power, electromagnetic, and acoustic

methods and studied how these could be implemented against embedded systems like Arduino and password-verification targets used in CSAW. Kian also researched how artificial intelligence and deep learning models could be integrated into side-channel workflows to automate trace alignment, classification, and key recovery. In addition to research, he worked directly on the qualification challenge, setting up and testing analysis scripts, reviewing provided traces, and experimenting with timing-based leak detection. He analyzed how early-exit behavior and busy-wait loops caused measurable side-channel leakage and outlined how these patterns could be exploited or mitigated. Beyond experimentation, Kian contributed extensively to the writing, editing, and structuring of the article. He refined the Methodology, Deep Learning Applications, and Variable Leaks sections, verified technical accuracy, and located supporting citations from IEEE and Springer sources. His combined work on AI-assisted analysis, documentation and qualification challenge experimentation provided the basis for the team’s technical summary and attack methodology.

B. Lindsay Estrella

Lindsay contributed in environment preparation, hardware operation, challenge analysis and documentation for the project. She began by learning through trial and error how to establish a functional ChipWhisperer environment, resolving many issues including installation and configuration errors. Through this process, she developed a strong understanding of the CW-Nano’s trigger behavior and general workflow.

Her primary analytical contributions were centered on the GateKeeper, Critical Calculations, and Ghost Blood challenges. For the GateKeeper challenge she examined the structure of the provided firmware and identified where timing and leakage behaviors emerged.

In the Critical Calculation challenge, Lindsay conducted baseline diagnostic, analyzed the timing of the device’s integrity-check loop, and implemented parameter-sweeping scripts to explore glitch offsets and widths. She documented the challenge’s clock alignment and glitch window sensitivity and ultimately revealed the flag by finding the right conditions. For Ghost Blood, she performed the initial investigative work, including exploring the obfuscation scheme and researching ROTL based encoding approaches.

C. David Kim

David focused on analyzing the verification logic used in the qualification challenge and identifying where timing and power leakage occurred. He examined how the comparison routine, the data-dependent delay behavior, and the matched-character counter contributed to observable side-channel signals, and outlined how these patterns could be used to build timing-based and fault-injection attack strategies. His work also included reviewing how the trigger signals defined a sensitive interval and how this could support trace alignment during analysis. After qualification, David continued working on the main GateKeeper challenge by studying the verifier’s structure,

evaluating how its timing behavior could be exploited in practice, and considering the impact of noise and early-exit behavior on recovery attempts. He also worked on the Sorter and SortersSong challenges, examining how the insertion-sort routine leaked information through differences in sorting effort and helping design an approach for recovering the hidden array values. For the Reverse challenge, he analyzed the transformation logic applied to inputs and mapped out how the underlying comparison process could be reconstructed. Alongside the technical work, David contributed to writing and organizing the team's reports. He updated the abstract and introduction, completed the Methodology and Scope sections, and ensured that explanations of leakage and attack strategy aligned with the observed behavior of the challenge binaries.

D. Tony Tanory

Tony has spent the semester researching side-channel and fault injection attacks, and working on capturing the Gatekeeper and Sorters Song Flags. After the release of the Gatekeeper qualification challenge, Tony performed a technical analysis and evaluation on the GateKeeper password verifier to identify and quantify the information leakage. Once the challenge began, the team found a potential flag for the first Gatekeeper challenge. Tony confirmed the flag with the ChipWhisperer hardware, and attempted a power-analysis attack to reveal the second flag, but it was unsuccessful.

For Sorter's Song 1, Tony developed my first full power-based side-channel attack against the ChipWhisperer firmware. The device hides a sorted array internally and uses insertion sort whenever my input is processed. By repeatedly sending controlled inputs and capturing power traces, he observed that the sorting routine took measurably longer whenever an inserted value caused additional element shifts. This allowed the team to fully reconstruct the secret 8-bit array and validate it with the device to obtain the flag.

The next Sorter's Song challenge, Tony extended this approach to a more difficult version of the challenge involving 16-bit values and more timing noise. Tony built a more robust analysis pipeline that smoothed the power-trace envelope and measured when the sorting activity returned to baseline, giving a consistent estimate of sorting duration. He added logic to detect duplicate values and performed binary search from right to left to recover all fifteen 16-bit array elements. Once the array was reconstructed, the values were converted into the required 30-byte little-endian payload, which the device accepted to reveal the second hidden flag. Together, these attacks demonstrated side-channel extraction against two increasingly complex challenges.

CONCLUSION

Over the duration of the semester, the work conducted for CSAW ESC achieved several objectives including the technical mastery of the ChipWhisperer environment, exploration of embedded vulnerabilities through hands-on experimentation, and critical evaluation of AI/LLMs as tools for enhancing and accelerating security workflows. Early engagement with

GateKeeper and other challenges provided a meaningful introduction to timing leakage, per-character delays, trigger-based measurement, and the development of automated scripts.

The middle phase of the semester culminated in a successful fault-injection attack on Critical Calculation and Sorters Song. Overall, the semester's work reflects a comprehensive engagement with embedded-systems security, through foundational setup and methodological learning, iterative challenge exploration, to integration of AI-powered reasoning. These efforts illustrate not only the technical depth of the CSAW ESC challenges but also the evolving role of AI tools as strategic assets in hardware security research.